Casey Barton

Honolulu, HI · (808) 936-5654 · glacier.equilibrium@gmail.com · github.com/GlacierEQ · casey-barton-glaciereq.vercel.app

PROFILE

I build governed multi-agent systems and the infrastructure they need to survive production: orchestration with typed handoff contracts, agent evaluation as release discipline — eval sets, behavioral assertions, LLM-as-judge rubrics, regression gates — and observability, monitoring, guardrails, and governance wired into every run. Secure, reliable, scalable, observable, governed: that is how I ship agents.

Three years as a founder building applied-AI systems from Honolulu: **65** MCP tool/API integrations, multi-agent systems orchestrating tools, APIs, and enterprise services, eval infrastructure that blocks regressions before production, and fail-closed guardrails — tracing, human-in-the-loop approvals, safe rollout, rollback as default. Density over duration: the systems below are built, running, and used daily, not claimed.

CORE COMPETENCIES

Engineering: TypeScript · Python · Node.js · REST APIs & microservices · containers · Supabase · Postgres (incl. pgvector) · SQL · Bash · GitHub Actions (CI/CD) · packaging & deployment · production diagnostics

Agentic Systems: multi-agent orchestration · typed handoff contracts · tool/function calling · planning & reasoning workflows · MCP protocol (**65** integrations) · enterprise API integration · RAG over vector databases (pgvector, Pinecone) & knowledge graphs (Neo4j) · prompt engineering

Evals, Observability & Governance: eval-set definition & behavioral assertions · LLM-as-judge with written rubrics · eval-driven prompt/tool/workflow iteration · regression gates & release thresholds · trace intelligence · monitoring & alerting posture · guardrails & policy controls · human-in-the-loop approvals · fail-closed recovery · audit receipts & decision logs · architecture & investment prioritization

EXPERIENCE

Founder / Applied AI Systems Architect 2023 – Present

GlacierEQ — Honolulu, HI

- Design and operate governed multi-agent systems: autonomous and semi-autonomous agents that plan, reason, decide, and execute, orchestrating interactions between agents, tools, APIs, and services through **65** MCP integrations — every interaction typed by contract, receipted, and reconstructable from its trace.
- Build and run agent evaluations as release discipline: define eval sets, test behavior against written assertions, iterate on prompts, tools, and workflows — LLM-as-judge scoring with rubrics, replay loops, explicit release thresholds, and CI regression gates that block a regression from merging. The document forge itself fails closed on unapproved claims.
- Implement observability, monitoring, guardrails, and governance for production AI: trace intelligence logging prompts, tool calls, and decisions on every run; per-task latency instrumentation; decision audit logs; human-in-the-loop approval gates typed into state transitions; policy controls and fail-closed recovery — production AI meeting enterprise standards across a 1,398-repo estate.

- Deploy and diagnose production agentic systems: package and ship TypeScript/Node.js and Python services with CI/CD, progressive delivery, monitoring, and rollback as the default posture — strong diagnostics across the AI stack, root-cause quarantine instead of silent retries, circuit breakers and dead-letter capture.
- Build and deploy RAG solutions over federated enterprise knowledge: retrieval-grounded generation with provenance, fanning out across Mem0, Supermemory, Supabase pgvector, Pinecone, and Neo4j behind one MCP interface — content-hash dedup, typed capability contracts, structured outputs.
- Set applied-AI architecture and investment priorities across the estate: consequential tradeoffs among models, build-vs-buy, and risk controls driven by telemetry — **989-test** suite, **62/62** behavioral checks green, **20+** production repositories maintained — the judgment load this role asks for, exercised daily against real load.
- Optimize AI workflows for performance, scalability, reliability, and cost: failure-injection recovery patterns, bounded tool actions, recoverable runs, and cost-aware iteration on model calls — dogfooded daily by the agent campaign engine that runs my own operation end to end.

EDUCATION

University of Hawaii at Manoa 2016
Bachelor of Science

AWS Cloud Institute 2025
Cloud Computing program

Colorado State University
Undergraduate coursework

SELECTED PROOF

Job-App Helix — evaluation-gated multi-agent campaign engine

- Attributable engine with immutable evidence models, stable finding codes, and fail-closed **GO / NO-GO** decisions — the eval-set-and-release-threshold pattern BMC names.
- Verification gates on every agent action: no completion claim without independent provider readback; every run reconstructable from its trace.

GlacierEQ estate — governed multi-agent infrastructure

- Orchestration primitives with typed handoff contracts, **65** MCP tool integrations with evidence boundaries and audit receipts, and human-in-the-loop control — across a live-verified 1,398-repo estate.
- Per-task latency instrumentation and decision audit logs on agent runs — the observability substrate governance depends on.

SKILLS

Languages: TypeScript · Python · SQL · Bash · JavaScript / Node.js

Agentic Systems: multi-agent orchestration · tool/function calling · MCP protocol (**65** integrations) · RAG (pgvector, Pinecone, Neo4j) · prompt engineering · planning & reasoning workflows

Evals, Observability & Governance: eval sets & behavioral assertions · LLM-as-judge · regression gates · release thresholds · trace intelligence · monitoring · guardrails & policy controls · human-in-the-loop · fail-closed recovery · audit receipts

*Evidence boundary: portfolio systems are independent work. No claim of enterprise-customer deployment track record, BMC-internal production work, or certification. Agentic frameworks were hand-rolled (deterministic DAG scheduler, **62/62** green); LangGraph/LangChain/CrewAI not claimed.*